

Cryptography Engineering Report

Structure of Code

The Program starts in `main.rs`, where either the server, the client, or both are initialized. When starting the server in `google.rs`, a short reset-handling takes place to restart the server in case of an error. Then, the `inner` section of the server is started, which waits for a message containing an action parameter. Based on this action parameter, the server either initializes a login or a registration request. The client, on the other hand, is implemented in `alice.rs`. There, after a short reset-handling, the `inner` section of the client is started, where a login or a registration request is initialized based on the user's input.

In both cases, before starting the `inner` section, the connection is secured via PQTLS to provide authentication and an initial layer of security. The downside of PQTLS is that it is less efficient than other TLS implementations like KEMTLS, which means more resources are required to run it. On the other hand, the protocol flow of PQTLS is closer to traditional TLS and is more established than KEMTLS. This is important, because newer post-quantum protocols have not yet had the time to be proven as secure as older, pre-quantum protocols and are therefore more likely to be broken.

For the registration, the OPRF registration protocol (part of the OPAQUE protocol from lecture 11), was used, which stores the username, a random salt, the encrypted client key bundle and the server key bundle in a database (in this case, a HashMap). The reason for using this protocol is to prevent attackers with access to the database from reading the password by saving a salted, hashed and encrypted version of the password instead of a plaintext.

For the login request, the continuation of the OPAQUE protocol was used, which consists of an OPRF-Phase to provide login data and decrypt the encrypted client key bundle, an AKE-Phase to generate a session key and a Key-Confirmation-Phase to provide verification. This protocol provides strong security for user passwords, even in an insecure TLS connection.

For the communication following the login request, the Double Ratchet protocol was used to ensure forward and backward secrecy in an ongoing message exchange. Combining Double Ratchet with OPAQUE and PQTLS requires a significant number of resources and is less efficient than other similar protocols like KEMTLS or SCRAM. However, depending on the function of the remote server and the sensitivity of the messages, those resources are a fair trade-off for a higher level of security.

Implementations

In the following sections, the breakdown and implementation of PQTLS, OPAQUE and Double Ratchet are explained.

PQTLS breakdown

PQTLS is a modification of TLS to provide post-quantum security. Just like TLS, it starts with the client sending a random nonce parameter and a public key to the server. Unlike TLS, the public key is a post-quantum secure encapsulation key like the ML-KEM encapsulation key. This encapsulation key is used by the server to generate a shared secret and a ciphertext containing the shared secret. The server then generates two shared keys (K_1^C and K_1^S) using the shared secret

Cryptography Engineering Report

and sends its own random nonce and the ciphertext to the client. This finishes the first “*ClientHello + ClientKE*” and “*ServerHello + ServerKE*” Phases. The server then generates multiple authentication parameters like a signature, a mac and a certificate. With these parameters, the previously generated shared secret and the public keys from server and client, the server generates four more keys (K_2^C , K_2^S , K_3^C and K_3^S). Lastly the server sends the authentication parameters, encrypted with the key K_1^S to the client and finishes the “*ServerCert + ServerFinished*” Phase. The client uses the received ciphertext and its own secret key to decapsulate the shared secret and compute K_1^C and K_1^S . Using K_1^S , the client decrypts the second received message to gain access to authentication parameters from the server. Just like the server, the client uses the authentication parameter, the shared secret and the server’s and client’s public keys to generate K_2^C , K_2^S , K_3^C and K_3^S . Then, if the verification of the authentication parameters succeeds, the client generates its own authentication parameters (e.g. mac) and sends it, encrypted with K_1^C , to the server. This finishes the “*ClientFinished*” Phase. After the server decrypts and verifies the authentication parameters of the client, the shared keys K_3^C and K_3^S can be used for a secure communication.

PQTLS implementation

In the project PQTLS is implemented in the `pq_tls` function inside both the server and the client. First, the client generates a `nonce_c`, a `dk` (decapsulation key) and an `ek` (encapsulation key) and sends the `nonce_c` and `ek` to the server. This message is sent, without an encryption, because these variables do not reveal any information about an existing shared secret and only allow the recipient to calculate a new shared secret.

The server then calculates the ciphertext (`ct`) and `shared_key` after receiving `nonce_c` and `ek`. Additionally, the server also generates a `key_pair` for later verification. Next, the methods `key_schedule_1` and `key_schedule_2` from lecture 3 are called to calculate `k1_c`, `k1_s`, `k2_c` and `k2_s`. These are keys which will also be calculated by the client to provide a shared secret. For verification, the server generates a certificate (`cert`), a signature (`google_sign`) and a mac (`mac_s`). The certificate is provided by a Certificate Authority (CA) to prevent man-in-the-middle (MiM) attackers from impersonating the server and hijacking the communication. The signature is used to prove that the client is communicating with an entity possessing the secret key of the server and the mac is used to prove the client is communicating with an entity which knows the shared secret (in this case `k2_s`). This also prevents MiM attackers from manipulating messages, as shown in lecture 3. With these new values, the `key_schedule_3` method is called, to calculate `k3_c` and `k3_s` on the server side. Afterward, the server sends two messages to the client. The first message is unencrypted and contains `nonce_s`, `ct` and `verifying_key` and the second message contains `cert`, `google_sign` and `mac_s` and is encrypted with AEAD using the key `k1_s`. The encryption is used to prevent MiM attackers from reading and manipulating the verification information in the second message.

The client first receives `nonce_s`, `ct` and `verifying_key` from the server and generates the `shared_key` from the `ct` and `dk`. Using these values, the client generates the shared secrets `k1_c`, `k1_s`, `k2_c` and `k2_s` analogously to the server. Afterwards, the client receives the second message, containing `cert`, `google_sign` and `mac_s` and decrypts it using `k1_s`. Again, analogously to the server, `k3_c` and `k3_s` are calculated. To provide secure communication, the client verifies

Cryptography Engineering Report

the `cert`, `google_sign` and `mac_s` and only after that generates `mac_c` and sends it to the server encrypted with `AEAD` and `k1_c`. The client doesn't send a certificate, because ordinary users don't get certificates issued by CAs. This is not a problem though, because the client is the one who established the connection and does not provide a service that needs authentication. Similarly, the client doesn't send a signature, because the client's secret key doesn't require verification in this step. Only the shared secret needs to be verified.

In the final step, the server receives `mac_c`, decrypts it with `k1_c` and verifies it. As previously mentioned, this provides a secure connection and prevents MiM attackers from manipulating the message. This completes the PQTLS protocol.

OPAQUE breakdown

The OPAQUE protocol is a protocol to ensure password security, even in an insecure TLS connection and prevent offline attacks. It consists of three phases. The OPRF-Phase, the AKE-Phase and the Key-Confirmation-Phase. In the OPRF-Phase, the client generates a random value `a` and sends the user-provided password as $h(pw)^a$, bundled with the username to the server. The server then retrieves the salt, the server key bundle and the encrypted client key bundle from the database, which were stored during the registration process. Afterwards the server sends the $h(pw)^{a \cdot salt}$ and the encrypted client key bundle back to the client. The client uses $h(pw)^{a \cdot salt}$ in addition to the value `a` and the user-provided password to decrypt the client key bundle and get access to its private and public key bundle. This completes the OPRF phase. In the AKE-Phase, both client and server generate a new secret and public key and exchange the public keys. Using each other's public keys, their own secret key and the key bundle values from the OPRF-Phase, both calculate a shared secret. This completes the AKE-Phase. In the final Key-Confirmation-Phase, the server and the client generate authentication parameters like macs using the shared secret and exchange them. After both sides have verified the authentication parameters, the OPAQUE-protocol ends.

OPAQUE implementation

In the project, the OPAQUE protocol is implemented in the `login` method in both the client and the server. The first phase is the OPRF-Phase, during which the client generates a random value `a` and calculates $h(pw)^a$ using the user provided password `pw`. This value gets sent, bundled with the username provided by the user, to the server over PQTLS. This is done to prevent the server from knowing the plain password.

After the server receives the $h(pw)^a$ and the username, it looks up the username inside the database (implemented as a HashMap) to find the corresponding salt `s`, the encrypted client key bundle (`enc_client_keys`) and the server key bundle (`lsk_s`, `lpk_c` and `lpk_s`). These values were stored in this configuration in the database during the registration process, to prevent attackers with access to the database from reading the plain password of the user. If the user entered the wrong username, the server would trigger a reset during this phase. Afterwards the server sends $h(pw)^{a \cdot s}$ and the `enc_client_keys` over PQTLS to the client.

After the client receives the $h(pw)^{a \cdot s}$ and the `enc_client_keys`, it calculates $h(pw)^s = (h(pw)^{a \cdot s})^{a^{-1}}$ to determine `rw = H(pw, h(pw)^s)` and `rw_key = KDF(rw)`. During the registration process,

Cryptography Engineering Report

`enc_client_keys` were encrypted with the key $\text{KDF}(\text{H}(\text{pw}, \text{h}(\text{pw})^s))$, which can now be decrypted using `rw_key`. This was done to only allow entities that possess the password to reconstruct the encryption key. The salt `s` was added to prevent offline attacks on the hashed password. If the user entered the wrong password, the decryption would fail, triggering a reset. After decrypting `enc_client_keys`, the client gains access to its own secret key (`lsk_c`), its own public key (`lpk_c`) and the server's public key (`lpk_s`). This completes the OPRF-Phase.

The second phase is the AKE-Phase, which begins with the client choosing a random value `x` and sending $X = g^x$ to the server, while the server chooses a random value `y` and sends $Y = g^y$ to the client. The value `g` is the generator of the group on which the encryption is based on. This process mirrors the Diffie-Hellman key exchange, to generate a shared secret. Both messages are sent over PQTLS. After both sides have received their respective message, the client calculates $\text{SK} = \text{HKDF}(\text{lpk}_s^x, Y^x, Y^{\text{lsk}_c})$, while the server calculates $\text{SK} = \text{HKDF}(X^{\text{lsk}_s}, X^y, \text{lpk}_c^x)$ to generate a shared session key (`SK`). This gets repeated for every session, to prevent the long-term compromise of one session key.

The last phase is the Key-Confirmation-Phase, in which both sides generate a mac to authenticate themselves. This phase begins by calculating $(\text{kc}, \text{ks}) = \text{HKDF}(\text{SK}, (\text{"Key Confirmation"}))$, `mac_c = HMAC(kc, "Client KC")` and `mac_s = HMAC(ks, "Server KC")` on the both server and the client. It is important to mention that both sides generate both macs for later verification. The client then sends `mac_c` to the server, while the server sends `mac_s` to the client, both over PQTLS. Once both sides have received the corresponding mac, the verification starts. As mentioned in the PQTLS section, this proves that both parties know the shared secret `SK` and that the messages have not been manipulated. This completes the Key-Confirmation-Phase and, with it, OPAQUE protocol.

Double Ratchet breakdown

The Double Ratchet protocol is used to provide both forward and backward secrecy during the message exchange. The goal is to prevent an attacker, with access to a short-term key, from decrypting previous sent messages as well as all following messages. Double Ratchet starts with a pre-known shared secret and is composed of two parts, the Symmetric Ratchet providing forward secrecy and the DH Ratchet providing backward secrecy. The Symmetric Ratchet works by using a one-way function like KDF to constantly change the encryption key. This way, in case of a leak, no previous encryption keys can be recovered, and all previous sent messages stay secure. The DH Ratchet works by constantly rotating the ephemeral keys. This means, for every first message of a chain, a new short-term key is generated, and a new public key is added to the message. In case of leakage, this prevents all following messages from being decrypted. Combining both methods provides both forward and backward secrecy. However, using Double Ratchet requires a lot of computational resources.

Double Ratchet implementation

In the project the Double Ratchet protocol is implemented in an endless `loop` statement, while the iterations of are implemented in the `double_ratchet_iteration` method in both the client and the server. This is done because the conversation between client and server does not have a predetermined end and can run endlessly. The client initializes the protocol with $k_i = \text{sk}$, $Y_i = Y$ and

Cryptography Engineering Report

$x_i = x$, while the server initializes with $k_i = sk$, $X_i = X$ and $y_i = y$ from the previously finished OPAQUE-protocol.

After the user inputs a message, the client chooses a new random x_{i+1} and calculates $(rk_{i+1}, ck_0) = kdf_{rk}(rk_i, Y_i^{x_{i+1}})$ and $(ck_1, mk_1) = kdf_{ck}(ck_0)$. Here, rk_{i+1} is the root key, which will be used to initialize the next message iteration, and its calculation part is of the DH Ratchet, providing backward secrecy. ck_1 and mk_1 are the chain and message keys, which are part of the Symmetric Ratchet, proving forward secrecy and allowing the client to encrypt multiple messages in a chain. In this implementation, the chain length is always one because, within the scope of this project, the server is designed to only echo the client's message. Longer chain lengths are useful in more complex applications, like messenger apps, which allow users to send several messages in succession. For a more complex server implementation, a section of the code is highlighted where the client can implement longer message chains. After the calculation of mk_1 , the message of the user is encrypted with AEAD and mk_1 and then sent, bundled with x_{i+1} to the server.

After the server receives this message, it calculates $(rk_{i+1}, ck_0) = kdf_{rk}(rk_i, X_{i+1}^y)$ and $(ck_1, mk_1) = kdf_{ck}(ck_0)$. In case of a longer chain length, a section is again highlighted to implement a longer chain recovery (ck_i, mk_i) . After calculating mk_1 , the server decrypts the message of the user.

Then the server prepares its response analogously to the client. It begins by choosing a new random value y_{i+1} and calculating $(rk_{i+2}, ck_0) = kdf_{rk}(rk_{i+1}, X_{i+1}^{y_{i+1}})$ and $(ck_1, mk_1) = kdf_{ck}(ck_0)$. Here, rk_{i+2} is again the next root key and its calculation part of the DH Ratchet, while ck_1 and mk_1 are the chain and message keys and their calculation part of the Symmetric Ratchet. The server then encrypts its response with AEAD and mk_1 and sends it, bundled with $y_{i+1} = g^{y_{i+1}}$ to the client.

The client then receives the message and calculates $(rk_{i+2}, ck_0) = kdf_{rk}(rk_{i+1}, Y_{i+1}^{x_{i+1}})$ and $(ck_1, mk_1) = kdf_{ck}(ck_0)$, to decrypt the response with mk_1 .

In the final step, the client initializes the next iteration with $rk_i = rk_{i+2}$, $Y_i = Y_{i+1}$ and $x_i = x_{i+1}$, while the server initializes it with $rk_i = rk_{i+2}$, $X_i = X_{i+1}$ and $y_i = y_{i+1}$. This is done to provide continuous security over multiple iterations. This completes the Double Ratchet protocol.